Laboratorio Redis V2

Caché con Redis en una API FastAPI + MongoDB

Objetivos:

- Levantar una API en FastAPI conectada a MongoDB.
- Medir con Postman el tiempo de una consulta lenta.
- Agregar Redis como caché y comparar tiempos (de unos 2 segundos a unos pocos milisegundos).
- Cambiar el tiempo de vida de la caché (TTL) sin modificar el código.
- Repetir el proceso en un ejercicio con las tarifas de ARESEP.

Requisitos previos

- **Docker**, con el contenedor de **MongoDB** corriendo en el puerto `27017`.
- **Python 3.11 o superior**.
- **Git**.
- **VS Code** con las extensiones *Python*, *MongoDB for VS Code* y *Redis for VS Code* (la publicada por Redis).
- **Postman** (aplicación de escritorio).
- Cuenta de **GitHub**.

Verificación rápida desde una terminal:

```
docker ps
python --version
git --version
```

Notas generales:

- Todos los comandos se ejecutan en la **terminal integrada de VS Code** (menú *Terminal → New Terminal*).
- Lo que está entre `< >` se reemplaza por el valor correspondiente. Ejemplo: `<usuario-github>`.
- Cada paso indica el resultado esperado. Si no se cumple, no continuar y revisar la sección *Si algo falla*.
- En Mac/Linux, si `python` no funciona, usar `python3`.

Parte 1. Obtener el proyecto

Paso 1. Hacer un fork

- Abrir https://github.com/maovares/api_pasajeros_redis.
- Clic en **Fork**.

- **Desmarcar** la casilla *Copy the main branch only* para copiar todas las ramas.
- Clic en **Create fork**.
- Resultado esperado: el repositorio queda en el usuario propio, con las ramas `main`, `cache` y `solucion-tarifas`.

Paso 2. Clonar el fork

```
git clone https://github.com/<usuario-github>/api_pasajeros_redis.git
```

- Resultado esperado: mensaje `Cloning into 'api_pasajeros_redis'...` sin errores.

Paso 3. Abrir el proyecto en VS Code

```
code api_pasajeros_redis
```

Si el comando `code` no existe, usar *File → Open Folder* y seleccionar la carpeta.

- Resultado esperado: se ven las carpetas `app` y `datos`, y los archivos `requirements.txt` y `.env.example`.

Paso 4. Revisar las ramas

Abrir la terminal integrada (`Ctrl+``) y ejecutar:

```
git branch -a
```

- Resultado esperado: `* main`, `remotes/origin/cache` y `remotes/origin/solucion-tarifas`.

Contenido de cada rama:

- `main`: la API **sin** caché.
- `cache`: `main` más Redis.
- `solucion-tarifas`: el ejercicio final resuelto. No abrirla hasta intentar el ejercicio.

Parte 2. Cargar los datos en MongoDB

El archivo `datos/pasajeros.json` contiene 1061 registros de pasajeros movilizados (datos abiertos de ARESEP).

Paso 5. Identificar el nombre del contenedor de Mongo

```
docker ps
```

Copiar el valor de la columna **NAMES** de la fila de MongoDB (por ejemplo `mongo`). En los pasos siguientes se indica como `<mongo>`.

Paso 6. Copiar el archivo al contenedor

Desde la carpeta del proyecto:

```
docker cp datos/pasajeros.json <mongo>:/tmp/pasajeros.json
```

- Resultado esperado: `Successfully copied ...`

Paso 7. Importar los datos

Este comando crea la base de datos `aresep` y la colección `passengers`:

```
docker exec <mongo> mongoimport --db aresep --collection passengers --jsonArray --drop
--file /tmp/pasajeros.json
```

Si Mongo tiene usuario y contraseña, agregar al final: `--username <usuario> --password <clave> --authenticationDatabase admin`

- Resultado esperado: `1061 document(s) imported successfully. 0 document(s) failed to import.`

Paso 8. Verificar con la extensión de MongoDB

- Clic en el ícono de la hoja verde (MongoDB) en la barra lateral de VS Code.
- *Add Connection → Connect with Connection String*.
- Escribir `mongodb://localhost:27017` y conectar.
- Abrir `aresep → passengers`.
- Resultado esperado: **1061** documentos.

Parte 3. Preparar y ejecutar la API

Paso 9. Crear el entorno virtual

```
python -m venv venv
```

- Resultado esperado: carpeta nueva llamada `venv`. Si VS Code pregunta si se desea usar, aceptar.

Paso 10. Activar el entorno virtual

- **Windows (PowerShell):** `venv\Scripts\Activate.ps1`
- **Windows (CMD):** `venv\Scripts\activate.bat`
- **Mac / Linux:** `source venv/bin/activate`

Si PowerShell indica que la ejecución de scripts está deshabilitada, ejecutar una vez este comando y repetir el paso:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

- Resultado esperado: `(venv)` al inicio de la línea de la terminal.

Paso 11. Seleccionar el intérprete en VS Code

- `Ctrl+Shift+P` (Mac: `Cmd+Shift+P`).
- Escribir **Python: Select Interpreter** y presionar Enter.
- Seleccionar el que está dentro de la carpeta `venv`.

- Resultado esperado: la barra inferior de VS Code muestra la versión de Python seguida de `('venv')`.

Paso 12. Instalar las dependencias

```
pip install -r requirements.txt
```

- Resultado esperado: `Successfully installed fastapi ... pymongo ... uvicorn ...`

Paso 13. Crear el archivo de configuración

- **Windows:** `copy .env.example .env`
- **Mac / Linux:** `cp .env.example .env`

Abrir `.env` y verificar que contenga:

```
MONGO_URI=mongodb://localhost:27017
MONGO_DB_NAME=aresep
SLOW_QUERY_SECONDS=2
```

- `SLOW_QUERY_SECONDS=2` hace que la consulta tarde 2 segundos a propósito, para que la mejora de la caché sea visible.
- Si Mongo usa usuario y contraseña: `MONGO_URI=mongodb://<usuario>:<clave>@localhost:27017/?authSource=admin`
- El archivo `.env` no se sube a Git porque puede contener contraseñas.

Paso 14. Ejecutar la API

```
uvicorn app.main:app --reload
```

- Resultado esperado: al final de la salida aparece `MongoDB: conexión OK` y `Application startup complete.`

Mantener esta terminal abierta. Para los siguientes comandos, abrir **otra** terminal (botón **+** del panel) y activar el entorno virtual nuevamente (Paso 10).

Paso 15. Abrir la documentación

Abrir en el navegador http://127.0.0.1:8000/docs.

- Resultado esperado: Swagger con la sección **Pasajeros** y dos rutas: `GET /api/pasajeros/` y `GET /api/pasajeros/{codigo_ruta}`.

Paso 16. Probar una ruta

- Abrir `GET /api/pasajeros/{codigo_ruta}` y clic en **Try it out**.
- Escribir `214` y clic en **Execute**.
- Resultado esperado: código `200` y los ramales de la ruta 214.

Parte 4. Medir con Postman (sin caché)

Paso 17. Importar la API en Postman

- Abrir http://127.0.0.1:8000/openapi.json en el navegador (definición de la API en JSON).
- En Postman: **Import**, pestaña **Link**, pegar `http://127.0.0.1:8000/openapi.json` y confirmar.
- Resultado esperado: colección **API de Pasajeros** con las rutas.

Si las URLs empiezan con `{{baseUrl}}` y aparecen en rojo: clic en la colección, pestaña **Variables**, crear `baseUrl` con valor `http://127.0.0.1:8000` y guardar.

Paso 18. Medir `GET /api/pasajeros/`

- Abrir la petición y clic en **Send**.
- Observar el **Time** de la respuesta (arriba a la derecha).
- Enviarla 3 veces más.
- Resultado esperado: `200 OK` y un tiempo de **unos 2 segundos** en cada envío. Anotar el tiempo.

La consulta tarda 2 segundos porque el servicio simula una consulta pesada (`asyncio.sleep` en `app/services/pasajeros_service.py`).

Parte 5. Levantar Redis

Paso 19. Crear el contenedor

```
docker run -d --name redis-lab -p 6379:6379 redis:8
```

Si el nombre `redis-lab` ya existe, ejecutar `docker start redis-lab`.

Paso 20. Comprobar que responde

```
docker exec -it redis-lab redis-cli ping
```

- Resultado esperado: `PONG`

Paso 21. Conectar el cliente dentro de VS Code

- Clic en el ícono de **Redis** en la barra lateral (extensión *Redis for VS Code*).
- Seleccionar **Connect database**.
- Host: `localhost`, Port: `6379`, y guardar.
- Resultado esperado: base de datos conectada y vacía (0 keys).

Alternativa: **Redis Insight**, aplicación independiente (https://redis.io/insight/). También se puede ejecutar con Docker: `docker run -d --name redisinsight -p 5540:5540 redis/redisinsight:latest` y abrir http://localhost:5540.

Parte 6. Agregar la caché

Paso 22. Traer el código de la rama `cache`

En la segunda terminal, con `(venv)` activo:

```
git checkout main
git merge origin/cache
```

- Resultado esperado: `Fast-forward` y una lista de archivos, entre ellos `app/cache/redis.py` y `requirements.txt`.

Se usa `origin/cache` porque al clonar solo se crea la rama `main` de forma local.

Paso 23. Instalar lo nuevo

La rama `cache` agrega la librería `redis`:

```
pip install -r requirements.txt
```

- Resultado esperado: `Successfully installed redis ...`

Paso 24. Completar el `.env`

Agregar al final de `.env`:

```
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_DB=0
REDIS_TTL=45
```

`REDIS_TTL` indica los **segundos** que un dato permanece en la caché.

Paso 25. Entender el decorador

Abrir `app/cache/redis.py` y `app/services/pasajeros_service.py`. La caché se activa con una sola línea sobre la función:

```
@redis_cache(ttl_seconds=settings.redis_ttl)
async def get_all_rutas():
    ...
```

En cada llamada a la función, el decorador:

- Crea una **llave** con el nombre de la función y sus argumentos.
- Si la llave **existe en Redis**, devuelve el valor guardado sin ejecutar la función (*Cache HIT*).
- Si **no existe**, ejecuta la función (*Cache MISS*), guarda el resultado con tiempo de expiración y lo devuelve.

Si Redis se cae, la API sigue funcionando sin caché. La caché es una mejora, no una dependencia obligatoria.

Paso 26. Reiniciar la API

En la terminal de uvicorn, `Ctrl+C` y ejecutar de nuevo:

```
uvicorn app.main:app --reload
```

- Resultado esperado: `MongoDB: conexión OK` y `Redis: conexión OK`.

Parte 7. Probar la caché

Paso 27. Primera llamada

Enviar `GET /api/pasajeros/` en Postman.

- Resultado esperado: unos **2 segundos**. En la terminal de uvicorn: `Cache MISS get_all_rutas`.

Paso 28. Segunda llamada

Enviar la misma petición otra vez.

- Resultado esperado: **pocos milisegundos** (entre 5 y 50 ms). En la terminal: `Cache HIT get_all_rutas`.

Paso 29. Ver la llave en Redis

En la extensión de Redis, refrescar la conexión.

- Resultado esperado: una llave que empieza con `cache:`, de tipo String, con un **TTL** que baja en cada refresco (inicia en 45).

Paso 30. Ver la expiración

Esperar 45 segundos y enviar la petición otra vez.

- Resultado esperado: otra vez unos 2 segundos y `Cache MISS`.

Paso 31. Apagar Redis a propósito (opcional)

```
docker stop redis-lab
```

Enviar la petición otra vez.

- Resultado esperado: `200` (más lento, sin caché) y en la terminal una advertencia `Redis no disponible`.

Volver a encender con `docker start redis-lab`.

Parte 8. Automatizar la prueba con Postman

Paso 32. Limpiar la caché antes de medir

Para iniciar siempre desde cero, la API tiene una ruta de ayuda solo para el laboratorio. En Postman, enviar `DELETE /api/cache/`.

- Resultado esperado: `{"message": "Caché limpiada"}`

Paso 33. Crear la prueba de "menos de 1 segundo"

- Abrir `GET /api/pasajeros/`.
- Ir a la pestaña **Scripts**, sub-pestaña **Post-response** (en versiones anteriores se llama **Tests**).
- Pegar este código y guardar:

```
pm.test("Responde en menos de 1 segundo", function () {
    pm.expect(pm.response.responseTime).to.be.below(1000);
});
```

También se puede usar la IA de Postman con una instrucción como: "Agregar una prueba que falle si la respuesta tarda más de 1 segundo". Revisar el código generado antes de aceptarlo.

Paso 34. Probar la prueba

Con la caché limpia (Paso 32), enviar `GET /api/pasajeros/` dos veces y revisar la pestaña **Test Results**.

- Resultado esperado: la primera vez **FAIL** (fue MISS, 2 s) y la segunda vez **PASS** (fue HIT).

Paso 35. Crear el Runner con 100 llamadas

- Enviar `DELETE /api/cache/` (Paso 32).
- Clic derecho en la colección y seleccionar **Run**.
- Dejar marcada **solo** la petición `GET /api/pasajeros/`.
- **Iterations:** `100`. **Delay:** `200` ms.
- Clic en **Start run**.
- Resultado esperado: **1 fallo** (la primera llamada, que fue MISS) y 99 aciertos. Anotar el tiempo promedio.

El delay de 200 ms hace que la corrida dure unos 20 segundos. Sin delay, las 100 llamadas terminan tan rápido que la caché nunca expira.

Parte 9. Cambiar el TTL sin modificar el código

Paso 36. Bajar el TTL a 3 segundos

Cambiar el valor en `.env` y guardar:

```
REDIS_TTL=3
```

Paso 37. Reiniciar la API

La opción `--reload` solo vigila archivos `.py`, no el `.env`. Detener uvicorn (`Ctrl+C`) y ejecutarlo otra vez:

```
uvicorn app.main:app --reload
```

No se modifica ninguna línea de código.

Paso 38. Repetir el Runner

- Enviar `DELETE /api/cache/`.
- Ejecutar el Runner igual que en el Paso 35.
- Resultado esperado: entre **6 y 9 fallos** (en la prueba de referencia fueron 8). Cada vez que la llave expira a los 3 segundos, una llamada vuelve a tardar los 2 segundos.

Paso 39. Preguntas para discusión

- ¿Por qué cambia el número de fallos si el código es el mismo?
- ¿Qué pasa con las consultas que llegan justo cuando expira la llave? ¿Y si fueran 1000 al mismo tiempo? (Investigar *cache stampede*).
- ¿Qué datos conviene cachear y cuáles no? ¿Cómo se elige el TTL?
- Si el dato cambia en Mongo antes de que expire el TTL, ¿qué respuesta se obtiene?

Parte 10. Ejercicio: tarifas de ARESEP

Crear una rama nueva para el ejercicio:

```
git checkout -b mi-solucion-tarifas
```

Preparar el JSON

Paso 40. Obtener el JSON de tarifas

Descargarlo de https://aresep.go.cr/autobus/tarifas/. Si no está disponible, usar la copia del proyecto: `datos/tarifas_original.json` (4883 registros). Crear una copia llamada `datos/tarifas.json` y trabajar sobre ella.

Paso 41. Abrir y formatear

- Abrir `datos/tarifas.json` en VS Code.
- `Shift+Alt+F` (Mac: `Shift+Option+F`) para formatear.
- Resultado esperado: objetos con 15 atributos, como `"Código Ruta"` y `"Tarifa Regular (¢)"`.

Paso 42. Identificar los problemas

- **Llaves con espacios, tildes y símbolos**, como `"Km por Viaje (Promedio)"`. Son difíciles de consultar y de mapear en Python.
- **Números guardados como texto**, como `"630.00"`. No se pueden promediar ni ordenar como números.
- **Fechas como texto** `dd-mm-aaaa`, como `"22-02-2024"`. No se pueden comparar como fechas.

El objetivo es llegar a **JSON extendido de Mongo**: los números son números y las fechas se escriben `{"$date": "2024-02-22T00:00:00Z"}`.

Estandarizar con Buscar y Reemplazar

Abrir Buscar y Reemplazar con `Ctrl+H` (Mac: `Cmd+Option+F`). Para cada reemplazo: escribir el *Find*, escribir el *Replace* y presionar **Replace All** (`Ctrl+Alt+Enter`).

Paso 43. Renombrar las 15 llaves

Con las expresiones regulares **apagadas** (ícono `.*`). Copiar el texto tal cual, con comillas y dos puntos:

Find	Replace
`"Código Ruta":`	`"codigo_ruta":`
`"Nombre Ruta":`	`"nombre_ruta":`
`"Nombre Fraccionamiento":`	`"nombre_fraccionamiento":`
`"Km por Viaje (Promedio)":`	`"km_por_viaje":`
`"Tarifa Regular (¢)":`	`"tarifa_regular":`
`"Tarifa Adulto Mayor (¢)":`	`"tarifa_adulto_mayor":`
`"Resolución":`	`"resolucion":`
`"Fecha Resolución":`	`"fecha_resolucion":`
`"Gaceta":`	`"gaceta":`
`"Alcance":`	`"alcance":`
`"Fecha Gaceta":`	`"fecha_gaceta":`
`"Fecha Vigencia":`	`"fecha_vigencia":`
`"Expediente":`	`"expediente":`
`"Operadores":`	`"operadores":`
`"Cédulas":`	`"cedulas":`

- Resultado esperado: cada reemplazo encuentra **4883** resultados.

Paso 44. Convertir los números de texto a número

Encender las expresiones regulares (ícono `.*` o `Alt+R`).

- Find: `"(km_por_viaje|tarifa_regular|tarifa_adulto_mayor|gaceta|alcance)": "(\d+(?:\.\d+)?)"`
- Replace: `"$1": $2`
- Resultado esperado: `"tarifa_regular": 630.00`, sin comillas en el valor.

Paso 45. Convertir las fechas

Con las expresiones regulares encendidas:

- Find: `"(fecha_resolucion|fecha_gaceta|fecha_vigencia)": "(\d{2})-(\d{2})-(\d{4})"`
- Replace: `"$1": {"$date": "$4-$3-$2T00:00:00Z"}`
- Resultado esperado: `"fecha_vigencia": {"$date": "2024-02-28T00:00:00Z"}`

Paso 46. Validar y guardar

- Buscar `"codigo_ruta"` con regex apagado: deben ser **4883** resultados.
- Guardar con `Ctrl+S`.

La rama `solucion-tarifas` incluye un script que hace lo mismo con código (`python scripts/normalizar_tarifas.py`). Usarlo solo para comparar el resultado.

Importar y exponer en la API

Paso 47. Importar a Mongo

```
docker cp datos/tarifas.json <mongo>:/tmp/tarifas.json
docker exec <mongo> mongoimport --db aresep --collection tarifas --jsonArray --drop
--file /tmp/tarifas.json
```

- Resultado esperado: `4883 document(s) imported successfully.`
- En la extensión de MongoDB, `fecha_vigencia` debe aparecer como tipo **Date**.

Paso 48. Crear el modelo

Crear `app/models/tarifa_model.py` con una clase `TarifaModel` con un campo por atributo, tomando como guía `ruta_model.py`. Tipos: `str`, `float`, `int` y `datetime`.

Paso 49. Crear el servicio

Crear `app/services/tarifas_service.py` con una función `get_all_tarifas()` que devuelva `await db.tarifas.find().to_list()`, tomando como guía `pasajeros_service.py`.

Paso 50. Crear la ruta

Crear `app/routes/tarifas.py` con `GET /` (prefijo `/tarifas`) y registrarla en `app/main.py` con `app.include_router(...)`.

- Resultado esperado: en `/docs` aparece la sección **Tarifas** y al ejecutarla devuelve `200` con las tarifas.

Paso 51. Agregar la caché

Agregar `@redis_cache(ttl_seconds=settings.redis_ttl)` sobre `get_all_tarifas` y comparar el MISS y el HIT en Postman.

La mejora es pequeña: Mongo responde esta consulta casi al instante y la mayor parte del tiempo se va en enviar 3 MB de JSON. La caché aporta más cuando la consulta es costosa, como en el Paso 53.

La consulta combinada

Paso 52. Tarifas por código de ruta

Crear `GET /api/tarifas/{codigo_ruta}`. Pistas:

- Una ruta tiene varias resoluciones a lo largo del tiempo. Se necesitan solo las **vigentes**, que son las de mayor `fecha_vigencia`.

- Buscar primero la última fecha con `find_one({"codigo_ruta": c}, sort=[("fecha_vigencia", -1)])` y luego traer todos los documentos con esa fecha.
- Si no hay resultados, lanzar `HTTPException(404)`.

Paso 53. Resumen: tarifas y promedio de pasajeros del año anterior

Crear `GET /api/tarifas/{codigo_ruta}/resumen?anno=2024` que devuelva:

- Las tarifas vigentes de la ruta.
- El **promedio de pasajeros** (regulares más adulto mayor) por ramal en el año `anno`.

Pistas:

- El promedio se calcula en Mongo con un *aggregation pipeline*: `$match` (ruta y año) y luego `$group` con `$avg` de `$add` de los dos campos de pasajeros.
- Con PyMongo async: `await (await db.passengers.aggregate(pipeline)).to_list()`.
- Los datos de pasajeros cargados son de **2024** y las tarifas vigentes son de 2025, por eso 2024 es el año anterior y el valor por defecto.
- Declarar esta ruta **antes** de `/{codigo_ruta}`, de lo contrario FastAPI usa la equivocada.

Paso 54. Cachear el resumen

Agregar también `@redis_cache` a esa función. En Redis se crea **una llave distinta por cada ruta y año**, porque la llave incluye los argumentos.

Resultado esperado para la ruta `214` (recortado):

```
{
  "codigo_ruta": "214",
  "nombre_ruta": "SAN JOSE-ZARCERO-LAGUNA DE ALFARO RUIZ",
  "anno_pasajeros": 2024,
  "tarifas_vigentes": [
    { "nombre_fraccionamiento": "SAN JOSE-NARANJO", "tarifa_regular": 1550.0,
      "tarifa_adulto_mayor": 1165.0, "fecha_vigencia": "2025-10-06T00:00:00" }
  ],
  "ramales_considerados": 1,
  "promedio_pasajeros": 6167.0
}
```

Paso 55. Comparar con la solución

Solo después de intentarlo:

```
git diff mi-solucion-tarifas origin/solucion-tarifas
```

Si algo falla

Síntoma	Solución
`ServerSelectionTimeoutError` al arrancar	Mongo no está accesible. Verificar con `docker ps` que el contenedor publique el puerto 27017 y revisar `MONGO_URI` en `.env`.
`Authentication failed`	Mongo tiene usuario y contraseña. Usar `MONGO_URI=mongodb://<usuario>:<clave>@localhost:27017/?authSource=admin`.
`GET /api/pasajeros/` devuelve `[]`	Los datos se importaron a otra base o colección. Debe ser `aresep` y `passengers` (Paso 7).
`No module named 'redis'`	Falta instalar dependencias después del merge: `pip install -r requirements.txt` (Paso 23).
`Redis: NO disponible`	El contenedor está apagado. Ejecutar `docker start redis-lab` y reiniciar uvicorn.
Cambio en `REDIS_TTL` sin efecto	`--reload` no vigila el `.env`. Reiniciar uvicorn (Paso 37).
`port is already allocated` al crear Redis	Ya hay algo en el puerto 6379. Revisar con `docker ps -a`.
`Address already in use` al arrancar uvicorn	Otra aplicación usa el puerto 8000. Usar `--port 8001` y ajustar Postman.
Postman: `Could not get response`	La API no está corriendo. Revisar la terminal de uvicorn.
`git merge cache`: `not something we can merge`	Usar `git merge origin/cache`.
`(venv)` no aparece en PowerShell	Ejecutar `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.